SHOWSHELF LAB · FOR THE AFTERNOON TEACHER

Teacher guide: Day 10

Students work through the steps in the Lab on their own. Each step has an explanation, a task, two hints and an answer button, so most of the time they can get unstuck alone. This guide gives you every task, a correct answer and the mistakes students usually make, so you can help quickly.

How to help: first ask them to read the "Not yet" message and the red Console line (if any), then open a hint. Only point at the answer after that. In the Lab, **Help › Teacher view** shows the answer and these notes on every step.

Day 10: How real apps are built

A taste of React and of servers

This morning the teacher showed you a real **React** project and a real **Express** server running on his laptop. This afternoon you try both ideas yourself, in small steps.

In a real project this code lives in files on your computer (React) or on a server (Express), and you'd install tools first: Node.js, and then React or Express with `npm`. The Lab lets you try the ideas without that.

- **Part 1, steps 1 to 6: React.** You rebuild the ShowShelf grid out of **components**: a card, a list of cards, a My List button and a search box.
- **Part 2, steps 7 to 12: a server.** You write the **routes** of a small My List server with Express. A ready-made page talks to it with `fetch`, so you see your server working straight away.

Today is a taste. The goal isn't to master React or Express in one afternoon, but to recognise the ideas when you meet them again. Everything you already know (functions, objects, arrays, `filter`, `fetch`) is used in both.

Step 1. Your first React component ~15 min

Teaches: A React component is a function that returns JSX; `createRoot(...).render(<App />)` draws it.

TASK

1. Write a component called `App` that returns a main heading: `<h1>ShowShelf</h1>`.
2. Under it, add the line that draws `<App />` inside `#root`.

COMMON MISTAKES

- A small letter: `function app()` and `<app />`. React treats lowercase tags as HTML elements.
- Forgetting the render line, so nothing appears.
- Quotes around the JSX: `return "<h1>ShowShelf</h1>";` shows the tags as text.

NOTE

If the preview stays empty, open the Console first: JSX mistakes come with a line number. The Lab translates the JSX into plain JavaScript before running it, so `import` lines don't work here (and aren't needed).

A CORRECT ANSWER

```
// App.jsx: your React code.
// React, ReactDOM and the list of shows (const shows = [...]) are ready to use.

function App() {
  return <h1>ShowShelf</h1>;
}

ReactDOM.createRoot(document.getElementById("root")).render(<App />);
```

Step 2. JavaScript values inside JSX ~10 min

Teaches: `{ }` in JSX shows the result of JavaScript; a component returns one outer element.

TASK

Make `App` return this structure, with the classes of the ShowShelf stylesheet:

1. an outer `<main className="container">`
2. inside it, a `<section className="hero">` with your `<h1>ShowShelf</h1>`
3. under the heading, a paragraph that says **12 shows to choose from**, where the number comes from `{shows.length}`

COMMON MISTAKES

- Returning two elements side by side without one element around them.
- Forgetting the `( )` after `return` on a multi-line JSX, so `return` returns nothing.
- `class=` instead of `className=` (React warns in the Console).

A CORRECT ANSWER

```
// ... unchanged ...

function App() {
  return (
    <main className="container">
      <section className="hero">
        <h1>ShowShelf</h1>
        <p>{shows.length} shows to choose from</p>
      </section>
    </main>
  );
}

// ... unchanged ...
```

Step 3. A ShowCard component with props ~20 min

Teaches: Props are a component's inputs: `<ShowCard show={...} />`, received as `function ShowCard({ show })`.

TASK

1. In `ShowCard`, replace the two `TODO` comments with an `<h3>` that shows `show.name`, and a `<p className="rating">` that shows a star and `show.rating`, like ★ **8.8**. (`{/* ... */}` is how you write a comment inside JSX.)
2. In `App`, under the hero section (still inside `<main>`), add `<div className="grid">` with one card inside: `<ShowCard show={shows[0]} />`.

COMMON MISTAKES

- `show="shows[0]"` in quotes: that passes the text "shows[0]", not the object.
- `<h3>show.name</h3>` without curly brackets: the words appear as they are.
- `{name}` instead of `{show.name}`: the prop is the whole show object.

A CORRECT ANSWER

```
// ... unchanged ...
        <p>{shows.length} shows to choose from</p>
      </section>

      <div className="grid">
        <ShowCard show={shows[0]} />
      </div>
    </main>
  );
// ... unchanged ...

ReactDOM.createRoot(document.getElementById("root")).render(<App />);

// One card for one show. Its prop (input) is `show`: one object from the list.
function ShowCard({ show }) {
  return (
    <article className="card">
      <img className="poster" src="poster.svg" alt="" />
      <div className="card-body">
        <span className="badge">{show.genres[0]}</span>
        <h3>{show.name}</h3>
        <p className="rating">★ {show.rating}</p>
      </div>
    </article>
  );
}
```

Step 4. A card for every show ~10 min

Teaches: `array.map((item) => <Comp key={item.id} ... />)` draws a list; each item needs a unique `key`.

TASK

In `App`, replace the single `<ShowCard show={shows[0]} />` with a card for **every** show: use `shows.map(...)`, and give each card `key={show.id}`.

COMMON MISTAKES

- `shows.map((show) => { <ShowCard show={show} /> })`: curly brackets without `return`, so no cards.
- `show={shows[0]}` inside the map: twelve cards of the same show.
- `key={show.name}` works here, but an id is safer: names can repeat.

A CORRECT ANSWER

```
// ... unchanged ...

      <div className="grid">
        {shows.map((show) => <ShowCard key={show.id} show={show} />)}
      </div>
    </main>
// ... unchanged ...
```

Step 5. Remember something with state ~20 min

Teaches: `const [value, setValue] = React.useState(start)` gives a component memory; calling the setter redraws it.

TASK

Give every card a My List button that switches between + and ✓:

1. At the top of `ShowCard` (before `return`), create state: `const [inList, setInList] = React.useState(false);`
2. Under the `<img>`, add a `<button>` with `className="list-btn"` and `aria-pressed={inList}`. When it's clicked, it calls `setInList(!inList)`.
3. The button's text is `{inList ? "✓" : "+"}`. (A normal `+` is fine too.)

`!inList` means "the opposite of inList": `true` becomes `false`, and `false` becomes `true`. Click a few buttons in the preview.

COMMON MISTAKES

- `onClick={setInList(!inList)}` without `() =>`: it runs while drawing, again and again ("Too many re-renders" in the Console).
- The `useState` line in `App` instead of `ShowCard`: all cards switch together.
- `onclick` with a small c: React ignores it and warns in the Console.

NOTE

Compare with the plain JavaScript version: there, the click handler had to find the button and change its text. Here the handler only changes state, and the JSX already says what each state looks like.

A CORRECT ANSWER

```
// ... unchanged ...
// One card for one show. Its prop (input) is `show`: one object from the list.
function ShowCard({ show }) {
  const [inList, setInList] = React.useState(false);

  return (
    <article className="card">
      <img className="poster" src="poster.svg" alt="" />
      <button className="list-btn" aria-pressed={inList} onClick={() => setInList(!inList)}>
        {inList ? "✓" : "+"}
      </button>
      <div className="card-body">
        <span className="badge">{show.genres[0]}</span>
// ... unchanged ...
```

Step 6. A search box that filters ~15 min

Teaches: A controlled input: `value={state}` and an `onChange` that sets the state; filter the data and map the result.

TASK

In `App`:

1. Create state for the search text: `const [search, setSearch] = React.useState("");`
2. In the hero, under the paragraph, add `<div className="search">` with a controlled `<input />`: `value={search}`, an `onChange` that calls `setSearch`, and a `placeholder` like "Search shows".
3. Before `return`, make a list of the shows whose name contains the search text (with `filter`, `toLowerCase` and `includes`), and `map` that list instead of `shows`.

Type `the` in the preview's search box.

COMMON MISTAKES

- `value={search}` without `onChange`: the box can't be typed in.
- Still mapping `shows` instead of the filtered list, so nothing changes.
- Using `onChange={setSearch}`: that saves the whole event, not the text. Use `(e) => setSearch(e.target.value)`.

NOTE

If a student asks why the My List ticks disappear when a card is filtered out and comes back: the card's state goes away with the card. In the full app, the list lives in `App` and is passed down as a prop. That's a good question to end the React part on.

A CORRECT ANSWER

```
// ... unchanged ...

function App() {
  const [search, setSearch] = React.useState("");
  const found = shows.filter((show) => show.name.toLowerCase().includes(search.toLowerCase()));

  return (
    <main className="container">
// ... unchanged ...
        <h1>ShowShelf</h1>
        <p>{shows.length} shows to choose from</p>
        <div className="search">
          <input
            type="search"
            placeholder="Search shows"
            value={search}
            onChange={(e) => setSearch(e.target.value)}
          />
        </div>
      </section>

      <div className="grid">
        {found.map((show) => <ShowCard key={show.id} show={show} />)}
      </div>
    </main>
// ... unchanged ...
```

Step 7. Your first server route ~15 min

Teaches: A route: `app.get("/path", (req, res) => { res.json(...); })`.

TASK

Under `// Your routes go here`, add a route for `GET /api/hello` that answers with an object like `{ message: "Hello from the server!" }`.

When it works, the top of the page shows what your server said. Then try the **Try a request** box at the bottom of the preview: press **Send**.

COMMON MISTAKES

- `res.send("Hello")` instead of `res.json({ message: ... })`: the answer is plain text, not an object.
- An address without the first `/`, or a typo like `/api/helo`.
- Putting the route inside the `{ }` of `app.listen(...)`. It works, but routes belong above it.

NOTE

The pretend server is only a stand-in: it understands the same code as Express for today's features. If students ask where Node.js is: it's what the teacher used this morning in the terminal (`npm run dev`).

A CORRECT ANSWER

```
// ... unchanged ...
// Your routes go here

app.get("/api/hello", (req, res) => {
  res.json({ message: "Hello from the server!" });
});

app.listen(3000, () => {
// ... unchanged ...
```

Step 8. Send My List ~10 min

Teaches: `res.json(array)` sends a list; for now the data lives in the server's memory.

TASK

1. Above `app.listen`, create `let myList` with at least two shows. Each show is an object with a number `id` and a `name`, like `{ id: 1, name: "Friends" }`.
2. Add a route for `GET /api/mylist` that answers with `myList`.

The page's list fills up with your shows.

COMMON MISTAKES

- Quotes around the ids: `id: "1"` is text, not a number.
- A missing comma between the objects in the array.
- `res.json("myList")` in quotes: that sends the word, not the list.

A CORRECT ANSWER

```
// ... unchanged ...
});

// My List lives in the server's memory
let myList = [
  { id: 1, name: "Friends" },
  { id: 2, name: "Dark" },
];

app.get("/api/mylist", (req, res) => {
  res.json(myList);
});

app.listen(3000, () => {
  console.log("ShowShelf API is ready");
// ... unchanged ...
```

Step 9. Add a show with POST ~15 min

Teaches: `app.post` and `req.body` receive data; `res.status(201).json(item)` answers "Created".

TASK

Above `app.listen`, add a route for `POST /api/mylist` that:

1. makes a new show object with `id: Date.now()` and `name: req.body.name`
2. adds it to `myList` with `push`
3. answers with status `201` and the new show

Then add a show with the form in the preview.

COMMON MISTAKES

- `res.json(item).status(201)`: the answer has already gone with 200 before the status is set.
- `name: req.body` (the whole body) instead of `req.body.name`.
- Forgetting `myList.push(item)`: the page says "Added", but the show never appears.

A CORRECT ANSWER

```
// ... unchanged ...
});

app.post("/api/mylist", (req, res) => {
  const item = { id: Date.now(), name: req.body.name };
  myList.push(item);
  res.status(201).json(item);
});

app.listen(3000, () => {
  console.log("ShowShelf API is ready");
// ... unchanged ...
```

Step 10. Say no to bad data ~10 min

Teaches: Validation: refuse bad data with `return res.status(400).json({ error: "..." })`.

TASK

At the top of your POST route, refuse a request without a name: answer `400` with `{ error: "..." }` and a message of your own. Don't forget the `return`.

Then press **Add** with an empty box: the page shows your message.

COMMON MISTAKES

- No `return`: the show is still added, and the Console warns that the server tried to answer twice.
- `if (req.body.name)` without the `!`: now the good shows are refused.
- The `if` placed after `myList.push(item)`: too late.

A CORRECT ANSWER

```
// ... unchanged ...

app.post("/api/mylist", (req, res) => {
  if (!req.body.name) {
    return res.status(400).json({ error: "Please send the name of a show" });
  }
  const item = { id: Date.now(), name: req.body.name };
  myList.push(item);
// ... unchanged ...
```

Step 11. Remove a show with DELETE ~20 min

Teaches: `app.delete("/api/mylist/:id", ...)`: `Number(req.params.id)`, 404 when it's missing, 204 when it's done.

TASK

Above `app.listen`, add a route for `DELETE /api/mylist/:id` that:

1. turns `req.params.id` into a number
2. answers `404` with an `error` if no show in `myList` has that id (use `find`)
3. otherwise removes the show with `filter` and answers `204`

Then press **Remove** on a show in the preview.

COMMON MISTAKES

- Forgetting `Number(...)`: `"2" === 2` is false, so every show is "not found".
- `myList.filter(...)` without `myList =` in front: the filter makes a new list, but nobody keeps it.
- `const myList`: then `myList = ...` crashes with "Assignment to constant variable".

A CORRECT ANSWER

```
// ... unchanged ...
});

app.delete("/api/mylist/:id", (req, res) => {
  const id = Number(req.params.id);
  const found = myList.find((show) => show.id === id);
  if (!found) {
    return res.status(404).json({ error: "That show is not in My List" });
  }
  myList = myList.filter((show) => show.id !== id);
  res.status(204).end();
});

app.listen(3000, () => {
  console.log("ShowShelf API is ready");
// ... unchanged ...
```

Step 12. Read the request log ~10 min

Teaches: Status codes: 2xx success, 4xx a wrong request, 5xx a server problem; a server's `console.log` goes to its log.

TASK

1. In the preview, add a show, add an empty name, remove a show, and send `GET /api/nothing` with **Try a request**. Watch which codes appear in the Console.
2. In your POST route, right before the answer, add a `console.log` that prints the name of the new show, like `console.log("Added:", item.name);`. Add a show again, and find your message in the Console.

COMMON MISTAKES

- Putting the `console.log` in the GET route instead of POST.
- `console.log("item.name")` in quotes: it prints the words, not the show's name.

NOTE

A good discussion to end the day: which part of the ShowShelf you know is the page, and which part could be a server? (The page draws; TVmaze and My List are data a server could keep.) If there's time, let students break a route on purpose (`throw new Error("Oops")`) to see a 500.

A CORRECT ANSWER

```
// ... unchanged ...
  const item = { id: Date.now(), name: req.body.name };
  myList.push(item);
  console.log("Added:", item.name);
  res.status(201).json(item);
});
// ... unchanged ...
```

Extra 1. Shows from TVmaze with useEffect ~20 min

Teaches: `React.useEffect(() => { ... }, [])` runs once after the first draw: a good place for `fetch`.

TASK

In `App`:

1. Create state `tvShows` that starts as an empty array: `React.useState([])`.
2. Add a `React.useEffect` that fetches `https://api.tvmaze.com/shows?page=0`, turns every show into our shape with `data.map(toShow)`, and stores the result with `setTvShows`.
3. Filter `tvShows` instead of `shows`.

Without internet, the Lab answers with 12 saved sample shows, so the cards look the same as before, but now they come from `fetch`.

COMMON MISTAKES

- No `[]` at the end of `useEffect`: the page fetches again after every draw.
- `setTvShows(data)` without `toShow`: React can't show `rating` because it's an object.

NOTE

Online, TVmaze answers with about 240 shows, so the grid gets long. That's expected. The check replaces TVmaze's answer with three known shows for a moment, to see that the cards really follow the data.

A CORRECT ANSWER

```
// ... unchanged ...
function App() {
  const [search, setSearch] = React.useState("");
  const [tvShows, setTvShows] = React.useState([]);

  // Runs once, after the first draw
  React.useEffect(() => {
    async function load() {
      const response = await fetch("https://api.tvmaze.com/shows?page=0");
      const data = await response.json();
      setTvShows(data.map(toShow));
    }
    load();
  }, []);
  const found = tvShows.filter((show) => show.name.toLowerCase().includes(search.toLowerCase()));

  return (
    <main className="container">
// ... unchanged ...
  );
}

// TVmaze's shows look different from ours. This turns one of them into our shape.
function toShow(tv) {
  return {
    id: tv.id,
    name: tv.name,
    genres: tv.genres,
    rating: tv.rating.average,
  };
}
```

Extra 2. Ask for one show ~10 min

Teaches: `GET /api/mylist/:id` answers with one item, or 404.

TASK

Add a route for `GET /api/mylist/:id` that answers with the show that has that id, or `404` with an `error` if there's none.

Test it with **Try a request**: `GET /api/mylist/1`, then `GET /api/mylist/99`.

A CORRECT ANSWER

```
// ... unchanged ...
});

app.get("/api/mylist/:id", (req, res) => {
  const id = Number(req.params.id);
  const found = myList.find((show) => show.id === id);
  if (!found) {
    return res.status(404).json({ error: "That show is not in My List" });
  }
  res.json(found);
});

app.listen(3000, () => {
  console.log("ShowShelf API is ready");
// ... unchanged ...
```

Extra 3. Change a status with PATCH ~20 min

Teaches: `app.patch` changes part of an item; check allowed values with `includes`.

TASK

1. Give every show a `status`: add `status: "want"` to the shows in `myList` and to the new show in your POST route. The page then shows a menu next to each show.
2. Add a route for `PATCH /api/mylist/:id` that answers `400` if `req.body.status` isn't `"want"`, `"watching"` or `"watched"`, `404` if the id isn't in the list, and otherwise changes the show's status and answers with the show.

Then change a status with the menu in the preview.

A CORRECT ANSWER

```
// ... unchanged ...
// My List lives in the server's memory
let myList = [
  { id: 1, name: "Friends", status: "want" },
  { id: 2, name: "Dark", status: "want" },
];

// ... unchanged ...
    return res.status(400).json({ error: "Please send the name of a show" });
  }
  const item = { id: Date.now(), name: req.body.name, status: "want" };
  myList.push(item);
  console.log("Added:", item.name);
// ... unchanged ...
});

const STATUSES = ["want", "watching", "watched"];

app.patch("/api/mylist/:id", (req, res) => {
  if (!STATUSES.includes(req.body.status)) {
    return res.status(400).json({ error: "status must be want, watching or watched" });
  }
  const found = myList.find((show) => show.id === Number(req.params.id));
  if (!found) {
    return res.status(404).json({ error: "That show is not in My List" });
  }
  found.status = req.body.status;
  res.json(found);
});

app.listen(3000, () => {
  console.log("ShowShelf API is ready");
// ... unchanged ...
```